

Compte-Rendu de Travaux Pratiques

Modules TP3 & TP4 : Programmation Réseau TCP et Scalabilité

Ismaël Ahamada
Licence 3 Informatique

2 mars 2026

Objectif du Document

Ce rapport synthétise l'implémentation, les tests et l'analyse théorique de l'évolution d'un serveur de jeu Mastermind fonctionnant sur le protocole TCP. Il documente la transition d'une architecture mono-client basique (TP3) à une architecture multi-client robuste, asynchrone et capable de supporter de fortes charges (TP4).

Table des matières

1 Analyse de l'Architecture Initiale (TP3)

1.1 Le modèle Mono-Client Bloquant

Le serveur implémenté lors du TP3 reposait sur un modèle canonique mono-thread. Le cycle de vie du `ServerSocket` était structuré autour d'une boucle principale qui appelait la méthode `accept()`.

Problématique : Que se passe-t-il lorsque deux clients se connectent simultanément à ce serveur ?

Lorsqu'un premier client établit une connexion (TCP 3-way handshake réussi), la méthode `accept()` retourne un objet `Socket`. Le serveur entre alors dans une boucle métier (`while(!gagne)`) qui effectue des lectures bloquantes via `BufferedReader.readLine()`.

Si un deuxième client tente de se connecter durant cette phase métier, sa requête TCP SYN sera mise en attente dans la file (*backlog*) du système d'exploitation associée au port d'écoute. Le serveur, dont le seul thread d'exécution est monopolisé par le premier client, ne pourra pas appeler une nouvelle fois `accept()` pour dépiler cette connexion. Le deuxième client subira un déni de service temporaire (délai d'attente réseau) ou définitif (timeout) tant que la première partie n'est pas achevée.

1.2 Solutions Conceptuelles pour la Concurrency

Pour pallier cette limitation fondamentale des I/O synchrones, l'état de l'art propose trois paradigmes majeurs :

1. **Le Multi-threading (Un thread par client)** : Le thread principal est exclusivement dédié à l'acceptation des requêtes (`accept()`). Chaque nouveau `Socket` est délégué à un nouveau *Thread* de travail (*Worker Thread*) qui gérera le cycle de vie applicatif du client de manière indépendante.

2. **Les Pools de Threads (Thread Pooling)** : Pour éviter le surcoût (*overhead*) de création/destruction de threads et la saturation de la RAM, on instancie un nombre fixe de threads de travail qui piochent les sockets dans une file d'attente partagée (Pattern *Producer-Consumer*).
3. **Les I/O Asynchrones et le Multiplexage (NIO)** : Utilisation de sélecteurs système (ex : `select`, `epoll` sous Linux, ou l'API `java.nio`). Un seul thread scrute simultanément l'état de multiples canaux (Channels) et ne déclenche de traitement (via API événementielle) *que* lorsque des données sont physiquement prêtes à être lues ou écrites, supprimant tout blocage.

2 Implémentation de la Scalabilité (TP4 : Multi-thread)

Afin de permettre des parties simultanées, l'architecture a été refactorisée selon le modèle "Un thread par client".

2.1 Délégation par classe interne Runnable

La logique métier du jeu (génération du code aléatoire, calcul des occurrences, envois de réponses) a été encapsulée dans une classe `ClientHandler` implémentant l'interface `Runnable`.

```

1 while (true) {
2     // 1. Appel bloquant limite uniquement a l'établissement de la pile
   TCP
3     Socket clientSocket = serverSocket.accept();
4     System.out.println("Nouveau client connecte : " + clientSocket.
      getInetAddress());
5
6     // 2. Delegation immediate a un thread concurrent
7     new Thread(new ClientHandler(clientSocket)).start();
8 }

```

Listing 1 – Extrait de la boucle d'écoute du Serveur Multi-thread

3 Tests de Performances et Mesure de Charge

La validation d'une architecture concurrente requiert des scénarios de test extrêmes (Stress Testing). Nous avons conçu l'automate `IdleClient` et le lanceur paramétrable `Stress1` pour simuler n connexions virtuelles opérant une transaction complète (connexion → requête → réponse) chronométrée au moyen de l'horloge système haute précision (`System.nanoTime()`).

3.1 Analyse du Comportement sous Charge (Stress 1)

3.1.1 Cas A : Sans fermeture de connexion (Gouffre de ressources)

Si l'automate ne ferme pas son socket après la transaction, lancer `java Stress1 5000` force le serveur à instancier et maintenir en vie 5 000 threads simultanés effectuant des blocages I/O (*Idle waiting*).

- **Conséquences système** : Le planificateur de l'OS (*Scheduler*) s'effondre sous le nombre de changements de contexte (*Context Switches*). L'empreinte mémoire explose (Chaque pile d'appel de thread consommant environ 1MB par défaut). Le système atteint rapidement la limite des descripteurs de fichiers autorisés (erreur type `java.net.SocketException: Too many open files`).

3.1.2 Cas B : Avec fermeture immédiate (Robustesse)

Si le client obéit à une fermeture propre (`socket.close()`), le comportement du serveur change drastiquement, même pour les mêmes valeurs de charge (ex : `java Stress1 5000 true`).

- **Résilience** : Le taux de rotation (*Churn rate*) des threads et connexions est extrêmement rapide. Dès qu'un client déconnecte, le serveur détruit le thread et le descripteur de fichier et le libère au système (Garbage Collection). La concurrence réelle en un instant t reste faible, permettant au serveur d'écouler la rampe de charge très facilement.

3.2 Mesure de la Latence (Ping Applicatif)

Nous avons mesuré le temps de réponse Δt s'écoulant entre la soumission de la trame TCP par le client et la réception de la trame de jugement du serveur. Les résultats empiriques (extraits du fichier `latence.csv`) illustrent l'impact du `backlog` lors d'un pic de requêtes simultanées :

Nombre de Clients (n)	Latence Moyenne (ns)	Latence Moyenne (ms)
1	6 425 333	~ 6.42 ms
2	966 145	~ 0.96 ms
10	1 396 362	~ 1.39 ms
100	3 696 213	~ 3.69 ms
1000	6 019 224	~ 6.01 ms
5000	6 134 732	~ 6.13 ms

Interprétation : Pour $n = 1$, la latence initiale intègre le coût de démarrage (Warm-up de la JVM, compilation JIT des classes de socket). Ensuite, la latence croît proportionnellement (de ~1ms à ~6ms) avec le nombre d'appels `accept()` se chevauchant au niveau du thread principal.

4 Conclusion

La migration du serveur Mastermind vers une architecture Multi-thread a prouvé son efficacité pour gérer des connexions entrelacées de manière non-bloquante au niveau global. Les tests de charge soulignent la fragilité structurelle du paradigme "1 Thread = 1 Client" face aux fuites de connexions (connexions dormantes/inactives), ce qui justifie dans l'industrie l'utilisation de serveurs asynchrones de type NIO/Netty pour la vraie ultra-haute disponibilité.